Question 1: Hard-Margin Linear SVM

(a) Plot and support vectors

The support vectors are point 1, $(1, 1)$, from the positive class, and points 5 and 6, $(1, 0)$ and $(0, 1)$, from the negative class.

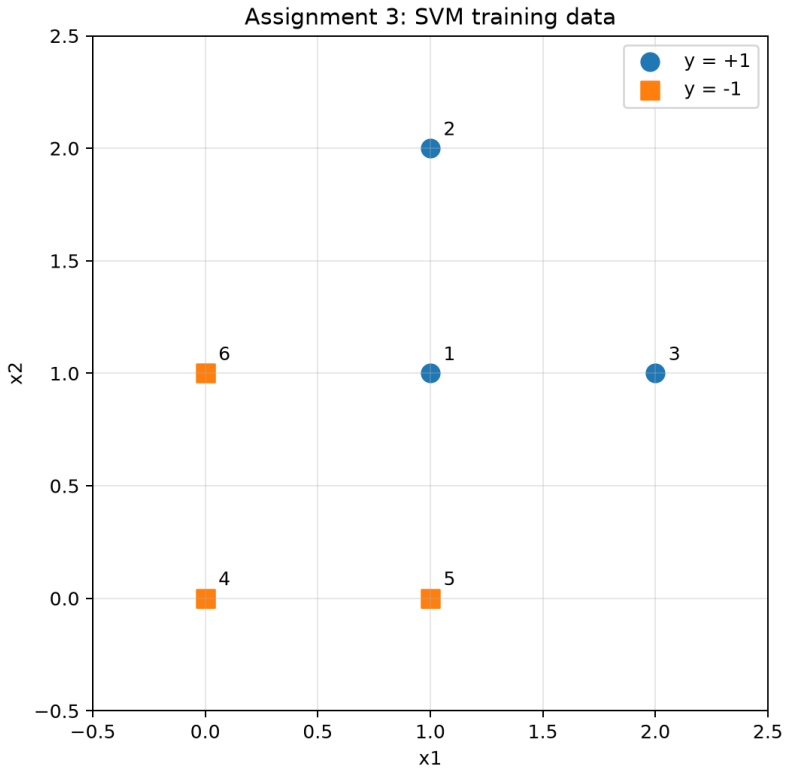


Figure 1: Training data with point IDs matching the assignment table.

By inspection, the closest parallel supporting lines are $x_1 + x_2 = 2$ for the positive class and $x_1 + x_2 = 1$ for the negative class. Points 1, 5, and 6 lie on these lines. The remaining points lie farther from the separating boundary.

(b) Separating boundary and geometric margin

For a hard-margin SVM in canonical form, the constraints are

$$y_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1,$$

with equality at the support vectors. Substituting points 1, 5, and 6 gives

$$w_1 + w_2 + b = 1, \tag{1}$$

$$w_1 + b = -1, \tag{2}$$

$$w_2 + b = -1. \tag{3}$$

Subtracting the second equation from the first gives $w_2 = 2$. Subtracting the third equation from the first gives $w_1 = 2$. Therefore, $b = -3$, and

$$\mathbf{w} = (2, 2)^T, \qquad b = -3.$$

The separating boundary is

$$x_2 = -x_1 + \frac{3}{2}.$$

To justify that this boundary maximizes the margin, consider the positive point $\mathbf{p} = (1, 1)$ and the negative-class convex-hull point

$$\mathbf{q} = \frac{(1,0) + (0,1)}{2} = \left(\frac{1}{2}, \frac{1}{2}\right).$$

Their distance is

$$\|\mathbf{p} - \mathbf{q}\| = \frac{1}{\sqrt{2}}.$$

Any linear separator with geometric margin γ must place these points on opposite sides at distances of at least γ from the boundary. Consequently,

$$2\gamma \leq \|\mathbf{p} - \mathbf{q}\| = \frac{1}{\sqrt{2}}.$$

Our boundary attains this upper bound and is therefore a maximum-margin separator.

The geometric margin, measured from the decision boundary to the closest training points, is

$$\gamma = \frac{1}{\|\mathbf{w}\|} = \frac{1}{\sqrt{2^2 + 2^2}} = \frac{1}{2\sqrt{2}} \approx 0.353553.$$

The full width between the two supporting boundaries is

$$2\gamma = \frac{1}{\sqrt{2}} \approx 0.707107.$$

As a check, the functional margins $y_i(\mathbf{w}^T\mathbf{x}_i + b)$, in point-ID order, are

$$(1, 3, 3, 3, 1, 1).$$

All constraints are satisfied, with equality exactly at points 1, 5, and 6.

Question 2: K-means

(a) Assignment and centroid updates

I implemented both update steps from scratch using NumPy.

For each data point $\mathbf{x}_i$, the assignment step selects the closest centroid:

$$c_i = \underset{j \in \{0,\ldots,k-1\}}{\arg\min} \|\mathbf{x}_i - \boldsymbol{\mu}_j\|_2^2.$$

Squared Euclidean distances give the same assignments as Euclidean distances because the square root is increasing. If multiple centroids are equally close, I choose the centroid with the smallest index, which is the behavior of `numpy.argmin`.

For each nonempty cluster, the centroid update computes the mean of its assigned points:

$$\boldsymbol{\mu}_j^{\text{new}} = \frac{1}{|C_j|} \sum_{i \in C_j} \mathbf{x}_i, \qquad C_j = \{i : c_i = j\}.$$

If a cluster is empty, I retain its previous centroid. The function creates a separate array rather than modifying the input centroids in place. This policy also handles duplicate initial centroids.

(b) Experiments and plots

I generated the dataset using

```
make_blobs(n_samples=4000, centers=4,
           cluster_std=0.60, random_state=0)
```

and ran K-means for $k = 2, 3, 4, 6, 10$. I used `numpy.random.default_rng(0)` and sampled initial centroids from the data with replacement, following the supplied initializer. For each value of k, I performed 100 assignment-and-update iterations, followed by a final assignment step. The true generating labels were not used during clustering.



Figure 2: K-means results for $k = 2$ and $k = 3$. Colors indicate cluster assignments; black crosses mark final centroids.



Figure 3: K-means results for $k = 4$ and $k = 6$.

With $k = 2$, the upper generating group forms one cluster, while the other three groups are largely merged. With $k = 3$, the upper and lower-right groups are separated, but the two middle groups are largely merged. With $k = 4$, the clusters closely match the four generating groups. Increasing k beyond four subdivides existing groups: in this run, $k = 6$ splits the left-middle group into three clusters, while $k = 10$ further subdivides the upper and lower-right groups. Thus, larger values of k produce finer partitions but do not necessarily better represent the four groups that generated the data. The particular subdivisions depend on initialization, since K-means can converge to a local minimum.

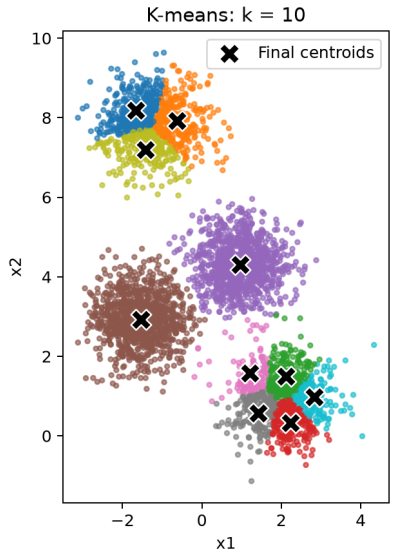


Figure 4: K-means results for $k = 10$.

Question 3: Noisy-Label SVM and K-NN Repair

(a) Noisy-label baseline

I generated independent training and test sets, each containing 1,000 positive examples and 1,000 negative examples:

$$\mathbf{x} \mid y = +1 \sim \mathcal{N}\left(\begin{bmatrix}1\\1\end{bmatrix}, I\right), \qquad \mathbf{x} \mid y = -1 \sim \mathcal{N}\left(\begin{bmatrix}-1\\-1\end{bmatrix}, 3I\right).$$

The training features, test features, and clean test labels were held fixed throughout the experiment.

For each trial, I independently corrupted the training labels using

$$P(\widetilde{y}_i = -y_i \mid y_i = +1) = 0.35, \qquad P(\widetilde{y}_i = -y_i \mid y_i = -1) = 0.20.$$

The flip probabilities were selected using the original clean labels, and each example received one random flip decision. Thus, no label could be flipped twice, and the original label array remained unchanged.

I trained an RBF SVM using

$$K(\mathbf{x}, \mathbf{z}) = \exp\left(-0.01\|\mathbf{x} - \mathbf{z}\|_2^2\right), \qquad C = 1.$$

Using `numpy.random.default_rng(0)`, I repeated label corruption and training 20 times. Each model was evaluated on the same clean test set. The mean baseline accuracy was

$$\overline{A}_{\text{baseline}} = 82.795\%.$$

(b) K-NN label repair

For each training point $\mathbf{x}_i$, I found its five nearest other training points using Euclidean distance. The point itself was explicitly excluded. Equal-distance ties were resolved by choosing smaller original training row indices first.

Let $N_5(i)$ denote these five neighbors. The repaired label is

$$\widehat{y}_i = \operatorname{sign}\left(\sum_{j \in N_5(i)} \widetilde{y}_j\right).$$

Since there are five binary votes, the sum cannot be zero. Every repaired label was computed from the same original noisy-label array, rather than from labels updated sequentially. The repair function used only training features and noisy training labels.

For each trial, I trained a fresh RBF SVM on the repaired labels using the same C and kernel parameters as the baseline. Both methods used the same training features, the same initial corrupted labels, and the same clean test set.

Method	Mean clean test accuracy
Noisy-label SVM	82.795%
Five-neighbor repair followed by SVM	86.440%

Table 1: Mean accuracy over 20 paired noise trials, using seed 0.

The difference in mean accuracy was

$$\overline{A}_{\mathrm{KNN}} - \overline{A}_{\mathrm{baseline}} = 0.864400 - 0.827950 = 0.036450,$$

which is an improvement of 3.645 percentage points.

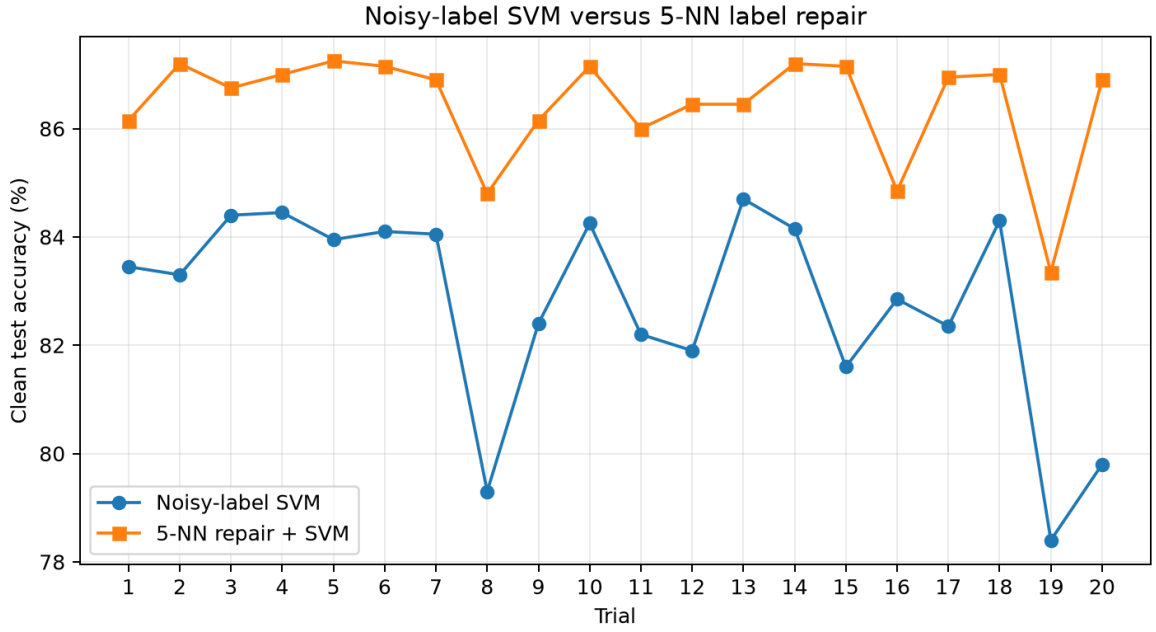


Figure 5: Clean test accuracy for each of the 20 paired trials. The final classifier is an RBF SVM for both methods; five-neighbor voting is used only to repair training labels.

K-NN repair improved test accuracy in all 20 trials in this experiment. A likely explanation is that nearby points often belong to the same underlying class, so majority voting can correct isolated label flips and provide cleaner training labels for the SVM. However, improvement is not guaranteed: the Gaussian classes overlap, and neighbors near the decision boundary may belong to different classes. A local majority can also be corrupted, causing the repair step to replace a correct label with an incorrect one. These results describe

repeated label corruption on one fixed training/test split, rather than variation across newly generated datasets.

The per-trial results are saved in `noise_comparison.csv`, with columns `trial`, `baseline_accuracy`, and `knn_accuracy`.